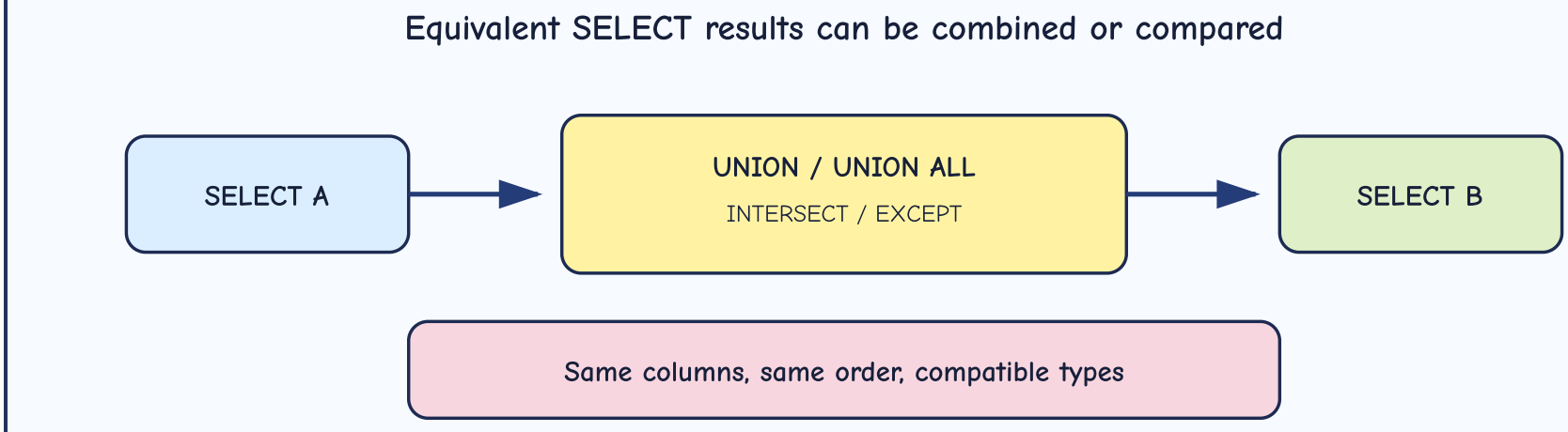
REVISION CHEAT SHEET:

# SQL Server Set Operators

INTERVIEW-READY NOTES - UNION - INTERSECT - EXCEPT - SQL - DIAGRAMS

DIAGRAM Set Operators Learning Map




The operator changes how the two row sets are interpreted - unique union, all rows, common rows, or left-only rows.

## TOPIC & SUBTOPIC INDEX

Study order: foundations -> structural rules -> UNION/UNION ALL -> INTERSECT/EXCEPT -> NULL/order/precedence -> practical patterns -> performance -> interview revision.


- Why Are They Called Set Operators?
- FundraisingDb Examples We Will Use
- Set Operators vs JOINs
- Core Requirements for All Set Operators
- Requirement 1 - Same Number of Columns
- Requirement 2 - Column Order Matters
- Requirement 3 - Compatible Data Types
- Requirement 4 - Result Column Names
- UNION
- UNION ALL
- UNION vs UNION ALL
- INTERSECT
- EXCEPT
- UNION, UNION ALL, INTERSECT, EXCEPT Together
- NULL Behavior in Set Operations
- ORDER BY with Set Operators
- Parentheses with Set Operators
- Set Operator Precedence
- Multiple UNION ALL Queries
- Set Operations with Aggregates
- UNION vs DISTINCT
- INTERSECT vs INNER JOIN
- INTERSECT vs EXISTS
- EXCEPT vs NOT EXISTS
- Set Operators with Different Schemas
- Data Type Precedence and Explicit CAST
- String Length and Precision
- Collation Considerations

- INTERSECT and EXCEPT Data-Type Restrictions
- Practical Pattern 1 - All Engaged Constituents
- Practical Pattern 2 - Complete Activity Timeline
- Practical Pattern 3 - Constituents Who Both Donated and Interacted
- Practical Pattern 4 - Prospects Who Have Never Donated
- Practical Pattern 5 - Funds Never Used
- Practical Pattern 6 - Campaigns with Donors Who Also Have Tasks
- Practical Pattern 7 - All Staff Participation IDs
- Practical Pattern 8 - Preserve Every Staff Activity
- Common Set Operator Mistakes
- Set Operator Performance Basics
- Set Operators Inside a CTE
- Decision Guide
- Master Set Operator Table
- Master Rules
- 20 Interview Questions and Query Problems
- Final Revision Map
- Final Easy Memory Rule


SQL Server **set operators** combine or compare the results returned by two or more **SELECT** queries.

The four important set operators are:

EXAMPLE / FLOW

```
UNION

UNION ALL

INTERSECT

EXCEPT
```

The easiest memory rule is:

EXAMPLE / FLOW

```
UNION
→ Combine rows and remove duplicates

UNION ALL
→ Combine rows and keep duplicates

INTERSECT
→ Keep rows common to both result sets

EXCEPT
→ Keep rows from the first result set
  that do not exist in the second
```

Microsoft SQL Server requires set-operation queries to return the same number of columns in the same order, with compatible corresponding data types. **UNION** removes duplicate rows while **UNION ALL** preserves them.

## Why Are They Called Set Operators?

Each **SELECT** query produces a result set.

For example:

EXAMPLE / FLOW

```
SELECT ConstituentId
FROM fundraising.Gift;
```

might produce:

EXAMPLE / FLOW

```
1001
1001
1002
1005
```

Another query:

EXAMPLE / FLOW

```
SELECT ConstituentId
FROM engagement.Interaction;
```

might produce:

EXAMPLE / FLOW

```
1001
1003
1005
```

Set operators allow us to ask questions such as:

EXAMPLE / FLOW

```
Which Constituents appear in either result?

Which appear in both?

Which appear only in the first?

Should duplicates be preserved?
```

## FundraisingDb Examples We Will Use

The main tables are:

EXAMPLE / FLOW

```
constituent.Constituent

fundraising.Gift

fundraising.Campaign

fundraising.Fund

fundraising.GiftAllocation

engagement.Interaction

engagement.Task

core.AppUser
```

The important relationships include:

EXAMPLE / FLOW

```
Constituent 1:M Gift

Campaign 1:M Gift

Gift M:N Fund
through GiftAllocation

Constituent 1:M Interaction

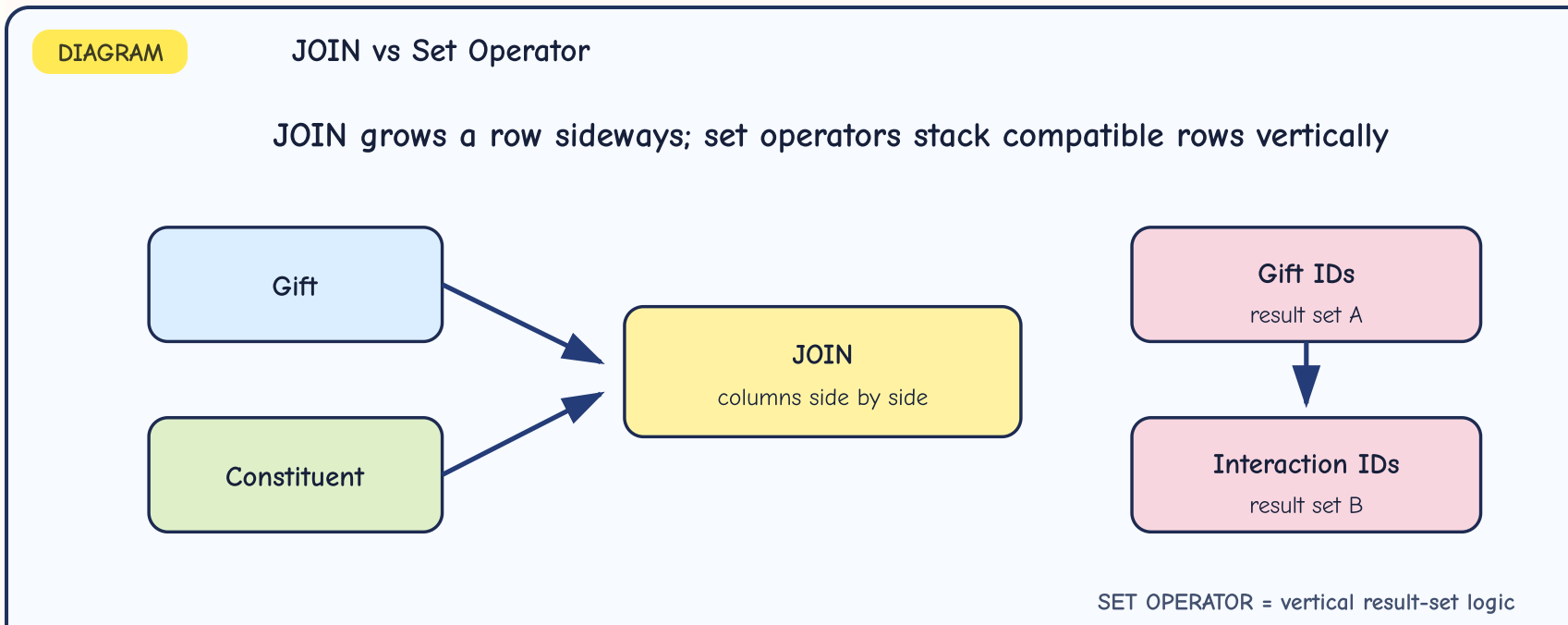
AppUser 1:M Interaction

Constituent 1:M Task

AppUser 1:M Task
```

## Set Operators vs JOINs

DIAGRAM




Use JOIN when you need related columns; use set operators when result shapes are equivalent.

This distinction is extremely important.

A **JOIN** combines related rows from different tables, usually side by side.

A **set operator** combines or compares entire query results, usually vertically.

## JOIN Example

Suppose we want:

> Show the Constituent name for every Gift.

EXAMPLE / FLOW

```
SELECT
    g.GiftId,
    c.DisplayName,
    g.TotalAmount
FROM fundraising.Gift AS g
INNER JOIN constituent.Constituent AS c
    ON c.ConstituentId = g.ConstituentId;
```

Result conceptually:

EXAMPLE / FLOW

```
GiftId | DisplayName | TotalAmount
```

Columns from two related tables are combined.

Think:

EXAMPLE / FLOW

```
JOIN
=
side-by-side relationship
```

## Set Operator Example

Suppose we want:

Combine Constituents who donated with Constituents who had an Interaction.

EXAMPLE / FLOW

```
SELECT ConstituentId
FROM fundraising.Gift

UNION

SELECT ConstituentId
FROM engagement.Interaction;
```

Both queries return the same type of thing:

EXAMPLE / FLOW

```
ConstituentId
```

The rows are combined vertically.

Think:

EXAMPLE / FLOW

```
SET OPERATOR
=
result set
on top of
result set
```

Microsoft distinguishes **UNION** from joins in exactly this way: **UNION** combines query result sets, while joins construct rows from columns belonging to related tables.

## Visual Difference

JOIN

EXAMPLE / FLOW

```
Gift                          Constituent
----                          -----------
GiftId                        ConstituentId
ConstituentId                 DisplayName
TotalAmount

          JOIN
           ↓

GiftId | DisplayName | TotalAmount
```

## Set Operation

EXAMPLE / FLOW

```
Gift ConstituentIds
-------------------
1001
1002
1005

Interaction ConstituentIds
--------------------------
1001
1003
1005

          UNION
            ↓

1001
1002
1003
1005
```

## Core Requirements for All Set Operators

Before using:

EXAMPLE / FLOW

```
UNION

UNION ALL

INTERSECT

EXCEPT
```

you must understand four important requirements.

EXAMPLE / FLOW

```
1. Same number of columns

2. Corresponding columns must be
   in the correct order

3. Corresponding data types
   must be compatible

4. Final output column names
   come from the first query
```

## Requirement 1 - Same Number of Columns

This is valid:

EXAMPLE / FLOW

```
SELECT
    ConstituentId,
    DisplayName
FROM constituent.Constituent

UNION

SELECT
    ConstituentId,
    CAST(Subject AS VARCHAR(200))
FROM engagement.Interaction;
```

Both queries return:

EXAMPLE / FLOW

```
2 columns
```

## Invalid Example

EXAMPLE / FLOW

```
SELECT
    ConstituentId,
    DisplayName
FROM constituent.Constituent

UNION

SELECT
    ConstituentId
FROM engagement.Interaction;
```

First query:

EXAMPLE / FLOW

```
2 columns
```

Second query:

EXAMPLE / FLOW

```
1 column
```

SQL Server cannot combine them.

## Easy Rule

If the first query returns:

EXAMPLE / FLOW

```
Column 1
Column 2
Column 3
```

every other branch must also return:

EXAMPLE / FLOW

```
Column 1
Column 2
Column 3
```

## Requirement 2 - Column Order Matters

Set operators compare columns by:

EXAMPLE / FLOW

```
POSITION
```

not by column name.

Suppose we want:

EXAMPLE / FLOW

```
PersonId
PersonName
```

This is logically correct:

EXAMPLE / FLOW

```
SELECT
    AppUserId,
    DisplayName
FROM core.AppUser

UNION ALL

SELECT
    ConstituentId,
    DisplayName
FROM constituent.Constituent;
```

Position mapping:

EXAMPLE / FLOW

```
Column 1
AppUserId      ↔ ConstituentId

Column 2
DisplayName    ↔ DisplayName
```

## Incorrect Column Order

EXAMPLE / FLOW

```
SELECT
    AppUserId,
    DisplayName
FROM core.AppUser

UNION ALL

SELECT
    DisplayName,
    ConstituentId
FROM constituent.Constituent;
```

Now SQL Server sees:

EXAMPLE / FLOW

```
Column 1:
AppUserId ↔ DisplayName

Column 2:
DisplayName ↔ ConstituentId
```

This is logically wrong and may also fail because of incompatible data types.

## Important Rule

Set operators do not ask:

> Do these columns have the same names?

They ask:

> Does column position 1 correspond to position 1, position 2 to position 2, and so on?

## Requirement 3 - Compatible Data Types

Corresponding columns must have compatible data types.

Example:

EXAMPLE / FLOW

```
SELECT
    ConstituentId
FROM constituent.Constituent

UNION

SELECT
    ConstituentId
FROM fundraising.Gift;
```

If both use compatible integer types such as:

EXAMPLE / FLOW

```
BIGINT
```

the operation is straightforward.

## What If Types Are Different?

SQL Server may perform an implicit conversion based on:

EXAMPLE / FLOW

```
data type precedence
```

Example conceptually:

EXAMPLE / FLOW

```
SELECT CAST(1001 AS BIGINT)

UNION

SELECT CAST(1002 AS INT);
```

SQL Server can convert the lower-precedence compatible numeric type.

## Dangerous Type Conversion

Suppose one branch returns:

EXAMPLE / FLOW

```
BIGINT
```

and another returns:

EXAMPLE / FLOW

```
VARCHAR
```

containing:

EXAMPLE / FLOW

```
'John Smith'
```

SQL Server may attempt a conversion that fails.

Therefore do not rely unnecessarily on implicit conversion.

Use explicit **CAST** when the intended result type is not obvious.

## Example - Explicit Conversion for an Activity Feed

Suppose we want to combine Interactions and Tasks into one result.

`InteractionDate` may be a datetime value while `Task.DueDate` may be a date.

We can normalize the output type explicitly:

EXAMPLE / FLOW

```
SELECT
    ConstituentId,
    CAST(InteractionDate AS DATETIME2) AS ActivityDate,
    'Interaction' AS ActivityType
FROM engagement.Interaction

UNION ALL

SELECT
    ConstituentId,
    CAST(DueDate AS DATETIME2),
    'Task'
FROM engagement.Task;
```

Now both branches intentionally return:

EXAMPLE / FLOW

```
BIGINT
DATETIME2
VARCHAR
```

or compatible equivalents.

## Requirement 4 - Result Column Names

The final result-column names come from the first query.

Example:

EXAMPLE / FLOW

```
SELECT
    AppUserId AS PersonId,
    DisplayName AS PersonName
FROM core.AppUser

UNION ALL

SELECT
    ConstituentId AS ConstituentNumber,
    DisplayName AS ConstituentName
FROM constituent.Constituent;
```

Final result names are:

EXAMPLE / FLOW

```
PersonId

PersonName
```

not:

EXAMPLE / FLOW

```
ConstituentNumber

ConstituentName
```

The aliases in later branches do not control the final result names.

## Best Practice

Put meaningful aliases in the first query.

Example:

EXAMPLE / FLOW

```
SELECT
    InteractionId AS ActivityId,
    ConstituentId,
    InteractionDate AS ActivityDate,
    'Interaction' AS ActivityType
FROM engagement.Interaction

UNION ALL

SELECT
    TaskId,
    ConstituentId,
    CAST(DueDate AS DATETIME2),
    'Task'
FROM engagement.Task;
```

Final column names:

EXAMPLE / FLOW

```
ActivityId
ConstituentId
ActivityDate
ActivityType
```

## UNION

### Simple Definition

`UNION` combines query results and removes duplicate complete rows.

Syntax:

EXAMPLE / FLOW

```
SELECT ...
FROM ...

UNION

SELECT ...
FROM ...;
```

## Fundraising Example - Constituents with Gifts or Interactions

Business question:

> Which Constituents either donated or had an Interaction?

EXAMPLE / FLOW

```
SELECT
    ConstituentId
FROM fundraising.Gift

UNION

SELECT
    ConstituentId
FROM engagement.Interaction;
```

Suppose Gift returns:

EXAMPLE / FLOW

```
1001
1001
1002
1005
```

and Interaction returns:

EXAMPLE / FLOW

```
1001
1003
1005
1005
```

`UNION` returns:

EXAMPLE / FLOW

```
1001
1002
1003
1005
```

Duplicates are removed.

## Why Is 1001 Returned Once?

Because `UNION` applies distinct semantics to the combined result.

Even if:

EXAMPLE / FLOW

```
Constituent 1001
```

has:

EXAMPLE / FLOW

```
10 Gifts
20 Interactions
```

the result contains:

EXAMPLE / FLOW

```
1001
```

only once.

## UNION Compares the Complete Selected Row

This is very important.

Suppose:

EXAMPLE / FLOW

```
SELECT
    ConstituentId,
    'Gift' AS SourceType
FROM fundraising.Gift

UNION

SELECT
    ConstituentId,
    'Interaction'
FROM engagement.Interaction;
```

For Constituent 1001, these rows are:

EXAMPLE / FLOW

```
1001 | Gift

1001 | Interaction
```

They are not duplicates because the complete rows differ.

Therefore both remain.

## Duplicate Means Entire Row

Given:

EXAMPLE / FLOW

```
1001 | Gift
```

and:

EXAMPLE / FLOW

```
1001 | Interaction
```

the first column matches.

But the second column does not.

Therefore the complete rows are different.

## When to Use UNION

Use `UNION` when:

EXAMPLE / FLOW

```
You need to combine result sets

AND

duplicate complete rows
should appear only once.
```

Fundraising examples:

EXAMPLE / FLOW

```
All Constituents who either donated
or had staff interaction

All AppUsers who participated
in several types of activity

Unique IDs gathered from multiple sources
```

## UNION ALL

### Simple Definition

`UNION ALL` combines result sets and preserves every row.

Duplicates are not removed.

## Fundraising Example - Unified Engagement Activity Feed

Suppose we want an activity stream consisting of:

EXAMPLE / FLOW

```
Interactions

Tasks
```

Every Interaction and every Task is a real separate event.

Therefore duplicates should not be removed.

EXAMPLE / FLOW

```
SELECT
    ConstituentId,

    CAST(InteractionDate AS DATETIME2)
        AS ActivityDate,

    'Interaction'
        AS ActivityType,

    Subject
        AS ActivityDescription

FROM engagement.Interaction

UNION ALL

SELECT
    ConstituentId,

    CAST(DueDate AS DATETIME2),

    'Task',

    Title
```

EXAMPLE / FLOW (CONT. 2)

```
FROM engagement.Task

ORDER BY ActivityDate DESC;
```

## Why UNION ALL Is Correct Here

Suppose John Smith has:

EXAMPLE / FLOW

```
Phone Interaction
on September 19
```

and:

EXAMPLE / FLOW

```
Call donor Task
on September 19
```

These are two legitimate business events.

Even if some selected values happen to look identical, we should not collapse them.

Therefore:

EXAMPLE / FLOW

```
UNION ALL
```

is the correct choice.

## UNION vs UNION ALL

Suppose Gift returns:

EXAMPLE / FLOW

```
1001
1001
1002
```

and Interaction returns:

EXAMPLE / FLOW

```
1001
1003
```

## UNION

EXAMPLE / FLOW

```
1001
1002
1003
```

## UNION ALL

EXAMPLE / FLOW

```
1001
1001
1002
1001
1003
```

## Easy Memory

EXAMPLE / FLOW

```
UNION
=
Combine + remove duplicates
```

EXAMPLE / FLOW

```
UNION ALL
=
Combine everything
```

## Performance Difference

`UNION ALL` generally requires less work because SQL Server does not have to determine which complete rows are duplicates.

`UNION` requires distinct processing.

Depending on the execution plan, duplicate removal can involve operations such as:

EXAMPLE / FLOW

```
Sort

Hash Match

Distinct processing

Memory usage
```

Therefore:

> If duplicate removal is not required, **`UNION ALL`** is usually the more appropriate operator.

But correctness comes first.

Do not use **`UNION ALL`** simply because it may be faster if duplicates would make the business result incorrect.

## Practical UNION Example - AppUsers Involved in Tasks

Your Task table references AppUser twice:

EXAMPLE / FLOW

```
CreatedByUserId
AssignedToUserId
```

The first tells us who created the Task.

The second tells us who is responsible for it.

Question:

> Which AppUsers have either created or been assigned at least one Task?

EXAMPLE / FLOW

```
SELECT
    CreatedByUserId AS AppUserId
FROM engagement.Task

UNION

SELECT
    AssignedToUserId
FROM engagement.Task;
```

A user who created 50 Tasks and was assigned 100 Tasks still appears once.

## Practical UNION ALL Example - Task Workload Events

Suppose instead we want to count **every role occurrence**.

EXAMPLE / FLOW

```
SELECT
    CreatedByUserId AS AppUserId,
    'Created' AS TaskRole
FROM engagement.Task

UNION ALL

SELECT
    AssignedToUserId,
    'Assigned'
FROM engagement.Task;
```

Now each business occurrence remains.

This result could later be aggregated:

EXAMPLE / FLOW

```
WITH TaskActivity AS
(
    SELECT
        CreatedByUserId AS AppUserId,
        'Created' AS TaskRole
    FROM engagement.Task

    UNION ALL

    SELECT
        AssignedToUserId,
        'Assigned'
    FROM engagement.Task
)

SELECT
    AppUserId,
    COUNT(*) AS ActivityCount
FROM TaskActivity
GROUP BY AppUserId;
```

## INTERSECT — Simple Definition

`INTERSECT` returns distinct rows that appear in **both** query results.

Think:

EXAMPLE / FLOW

```
Set A
AND
Set B
```

or mathematically:

EXAMPLE / FLOW

```
A ∩ B
```

SQL Server returns distinct common rows.

## Fundraising Example - Donors Who Also Had Interactions

Business question:

> Which Constituents have made at least one Gift and also have at least one Interaction?

First set:

EXAMPLE / FLOW

```
SELECT ConstituentId
FROM fundraising.Gift;
```

Second set:

EXAMPLE / FLOW

```
SELECT ConstituentId
FROM engagement.Interaction;
```

Combine with `INTERSECT`:

EXAMPLE / FLOW

```
SELECT
    ConstituentId
FROM fundraising.Gift

INTERSECT

SELECT
    ConstituentId
FROM engagement.Interaction;
```

## Example Result

Gift Constituents:

EXAMPLE / FLOW

```
1001
1001
1002
1005
```

Interaction Constituents:

EXAMPLE / FLOW

```
1001
1003
1005
```

Result:

EXAMPLE / FLOW

```
1001
1005
```

Why?

Because only those Constituents appear in both sets.

## INTERSECT Removes Duplicates

Even if:

EXAMPLE / FLOW

```
Constituent 1001
```

has:

EXAMPLE / FLOW

```
20 Gifts
50 Interactions
```

the output is still:

EXAMPLE / FLOW

```
1001
```

once.

`INTERSECT` returns distinct matching rows.

## Practical INTERSECT Example - Campaign Donors with Interactions

Suppose we want:

Constituents who donated to Campaign 200 and also had an Interaction.

EXAMPLE / FLOW

```
SELECT
    ConstituentId
FROM fundraising.Gift
WHERE CampaignId = 200

INTERSECT

SELECT
    ConstituentId
FROM engagement.Interaction;
```

The first query defines:

EXAMPLE / FLOW

```
Campaign-200 donors
```

The second query defines:

EXAMPLE / FLOW

```
Constituents with interactions
```

`INTERSECT` keeps only those found in both.

## INTERSECT with Multiple Columns

Suppose we compare:

EXAMPLE / FLOW

```
ConstituentId
AppUserId
```

pairs.

For example:

EXAMPLE / FLOW

```
SELECT
    ConstituentId,
    FundraiserUserId AS AppUserId
FROM engagement.Interaction

INTERSECT

SELECT
    ConstituentId,
    AssignedToUserId
FROM engagement.Task;
```

This asks:

> Which Constituent/AppUser pairs appear both as an Interaction relationship and as a Task assignment?

A match requires both values to correspond.

Example:

EXAMPLE / FLOW

```
1001 | 10
```

matches:

EXAMPLE / FLOW

```
1001 | 10
```

but not:

EXAMPLE / FLOW

```
1001 | 11
```

because the whole selected row is compared.

## EXCEPT — Simple Definition

EXCEPT returns distinct rows from the **left query** that are absent from the **right query**.

Think:

EXAMPLE / FLOW

```
A minus B
```

or:

EXAMPLE / FLOW

```
A - B
```

SQL Server returns distinct left-side rows not returned by the right-side query.

## Fundraising Example - Constituents Who Have Never Donated

Business question:

> Which Constituents exist but have no Gift?

Left set:

EXAMPLE / FLOW

```
All Constituents
```

Right set:

EXAMPLE / FLOW

```
Constituents appearing in Gift
```

Query:

EXAMPLE / FLOW

```
SELECT
    ConstituentId
FROM constituent.Constituent

EXCEPT

SELECT
    ConstituentId
FROM fundraising.Gift;
```

Result:

EXAMPLE / FLOW

```
Constituents with no Gift
```

This is a very practical fundraising query because a Constituent may be:

EXAMPLE / FLOW

```
donor

prospect

volunteer

board member

corporation

foundation
```

without having donated yet.

## EXCEPT Is Directional

This is extremely important.

EXAMPLE / FLOW

```
A EXCEPT B
```

is generally not equal to:

EXAMPLE / FLOW

```
B EXCEPT A
```

## Example

EXAMPLE / FLOW

```
SELECT ConstituentId
FROM constituent.Constituent

EXCEPT

SELECT ConstituentId
FROM fundraising.Gift;
```

means:

EXAMPLE / FLOW

```
Constituents with no Gift
```

Reverse it:

EXAMPLE / FLOW

```
SELECT ConstituentId
FROM fundraising.Gift

EXCEPT

SELECT ConstituentId
FROM constituent.Constituent;
```

This would conceptually ask:

EXAMPLE / FLOW

```
Gift ConstituentIds
that do not exist in Constituent
```

In a properly enforced database with a trusted foreign key, that result should normally be empty.

That makes the reverse query useful as a data-integrity check in legacy/import situations.

## Practical EXCEPT Example - Funds with No Allocations

Your model has:

EXAMPLE / FLOW

```
Fund
    |
    | 1:M
    v
GiftAllocation
```

Question:

Which Funds have not received any GiftAllocation?

EXAMPLE / FLOW

```
SELECT
    FundId
FROM fundraising.Fund

EXCEPT

SELECT
    FundId
FROM fundraising.GiftAllocation;
```

## Practical EXCEPT Example - Campaigns with No Gifts

EXAMPLE / FLOW

```
SELECT
    CampaignId
FROM fundraising.Campaign

EXCEPT

SELECT
    CampaignId
FROM fundraising.Gift
WHERE CampaignId IS NOT NULL;
```

This finds Campaigns that exist but currently have no Gift associated with them.

## Practical EXCEPT Example - Constituents with Gifts but No Interactions

EXAMPLE / FLOW

```
SELECT
    ConstituentId
FROM fundraising.Gift

EXCEPT

SELECT
    ConstituentId
FROM engagement.Interaction;
```

Business meaning:

Constituents who donated but do not yet have an Interaction recorded.

This could be useful when identifying donors who may need stewardship follow-up.

## UNION, UNION ALL, INTERSECT, EXCEPT Together

DIAGRAM

One Example, Four Results

Same two input sets - four different business answers

Gift Constituents

1001, 1001, 1002, 1005

Interaction Constituents

1001, 1003, 1005

UNION

1001,1002,1003,1005

UNION ALL

keep every occurrence

INTERSECT

1001,1005

EXCEPT

Gift - Interaction = 1002

Duplicate handling is part of the business requirement.

The selected row shape is the unit that each set operator compares.

Suppose:

EXAMPLE / FLOW

```
Gift ConstituentIds

1001
1001
1002
1005
```

and:

EXAMPLE / FLOW

```
Interaction ConstituentIds

1001
1003
1005
```

Then:

EXAMPLE / FLOW

```
UNION
→ 1001,1002,1003,1005
```

EXAMPLE / FLOW

```
UNION ALL
→ 1001,1001,1002,1005,1001,1003,1005
```

EXAMPLE / FLOW

```
INTERSECT
→ 1001,1005
```

EXAMPLE / FLOW

```
Gift EXCEPT Interaction
→ 1002
```

## NULL Behavior in Set Operations

NULL behavior is important.

Normally:

EXAMPLE / FLOW

```
NULL = NULL
```

does not evaluate to **TRUE**.

SQL uses three-valued logic.

However, for determining distinct rows in **EXCEPT** and **INTERSECT**, SQL Server treats two NULL values as equal.

## Fundraising NULL Example

Suppose some Gifts are not linked to a Campaign:

EXAMPLE / FLOW

```
CampaignId = NULL
```

and some comparison query also returns:

EXAMPLE / FLOW

```
NULL
```

When SQL Server determines duplicate/distinct rows for set operations, those NULL values can match for set-comparison purposes.

## UNION with NULL

Example:

EXAMPLE / FLOW

```
SELECT CampaignId
FROM fundraising.Gift

UNION

SELECT CampaignId
FROM fundraising.Gift;
```

If several Gift rows have:

EXAMPLE / FLOW

```
CampaignId = NULL
```

the final **UNION** result contains at most one distinct NULL row.

## UNION ALL with NULL

EXAMPLE / FLOW

```
SELECT CampaignId
FROM fundraising.Gift

UNION ALL

SELECT CampaignId
FROM fundraising.Gift;
```

All NULL occurrences remain.

`UNION ALL` does not deduplicate anything.

## INTERSECT with NULL

If NULL appears in both sets:

EXAMPLE / FLOW

```
SELECT CampaignId
FROM fundraising.Gift

INTERSECT

SELECT CampaignId
FROM fundraising.Campaign;
```

you must reason carefully about column definitions and actual data.

For the abstract set-comparison rule:

EXAMPLE / FLOW

```
NULL vs NULL
```

can be treated as equal for distinct-row determination.

## EXCEPT and NULL

If the left query contains a NULL-equivalent row and the right query contains the same NULL-equivalent row, it is removed from the `EXCEPT` result.

This differs from the surprises people sometimes see with:

EXAMPLE / FLOW

```
NOT IN
```

and NULL.

## ORDER BY with Set Operators

The final `ORDER BY` goes at the end of the complete set expression.

SQL Server allows top-level ordering after the combined set operation rather than independently ordering each branch.

## Correct Example

EXAMPLE / FLOW

```
SELECT
    ConstituentId,
    CAST(InteractionDate AS DATETIME2)
        AS ActivityDate,

    'Interaction'
        AS ActivityType

FROM engagement.Interaction

UNION ALL

SELECT
    ConstituentId,
    CAST(DueDate AS DATETIME2),
    'Task'

FROM engagement.Task

ORDER BY
    ActivityDate DESC;
```

The final `ORDER BY` sorts:

EXAMPLE / FLOW

```
the complete combined result
```

## Incorrect General Pattern

Do not write ordinary branch-level sorting like:

EXAMPLE / FLOW

```
SELECT ...
FROM engagement.Interaction
ORDER BY InteractionDate

UNION ALL

SELECT ...
FROM engagement.Task
ORDER BY DueDate;
```

That is not how final ordering of a top-level set operation works.

## Important ORDER BY Alias Rule

The final result names come from the first query.

Therefore:

EXAMPLE / FLOW

```
SELECT
    ConstituentId,
    InteractionDate AS ActivityDate
FROM engagement.Interaction

UNION ALL

SELECT
    ConstituentId,
    CAST(DueDate AS DATETIME2)
FROM engagement.Task

ORDER BY ActivityDate;
```

works because:

EXAMPLE / FLOW

```
ActivityDate
```

is defined by the first query.

SQL Server requires aliases used by a final **ORDER BY** with these set operators to be exposed in the select list, and the first query defines the output column names.

## ORDER BY Column Number

SQL Server also permits:

EXAMPLE / FLOW

```
ORDER BY 2;
```

meaning:

EXAMPLE / FLOW

```
sort by the second result column.
```

But this is less readable.

Prefer:

EXAMPLE / FLOW

```
ORDER BY ActivityDate;
```

## Parentheses with Set Operators

When several operators are combined, parentheses can control or clarify evaluation.

Example:

EXAMPLE / FLOW

```
(
    SELECT ConstituentId
    FROM fundraising.Gift

    UNION

    SELECT ConstituentId
    FROM engagement.Interaction
)

EXCEPT

SELECT ConstituentId
FROM engagement.Task;
```

Meaning:

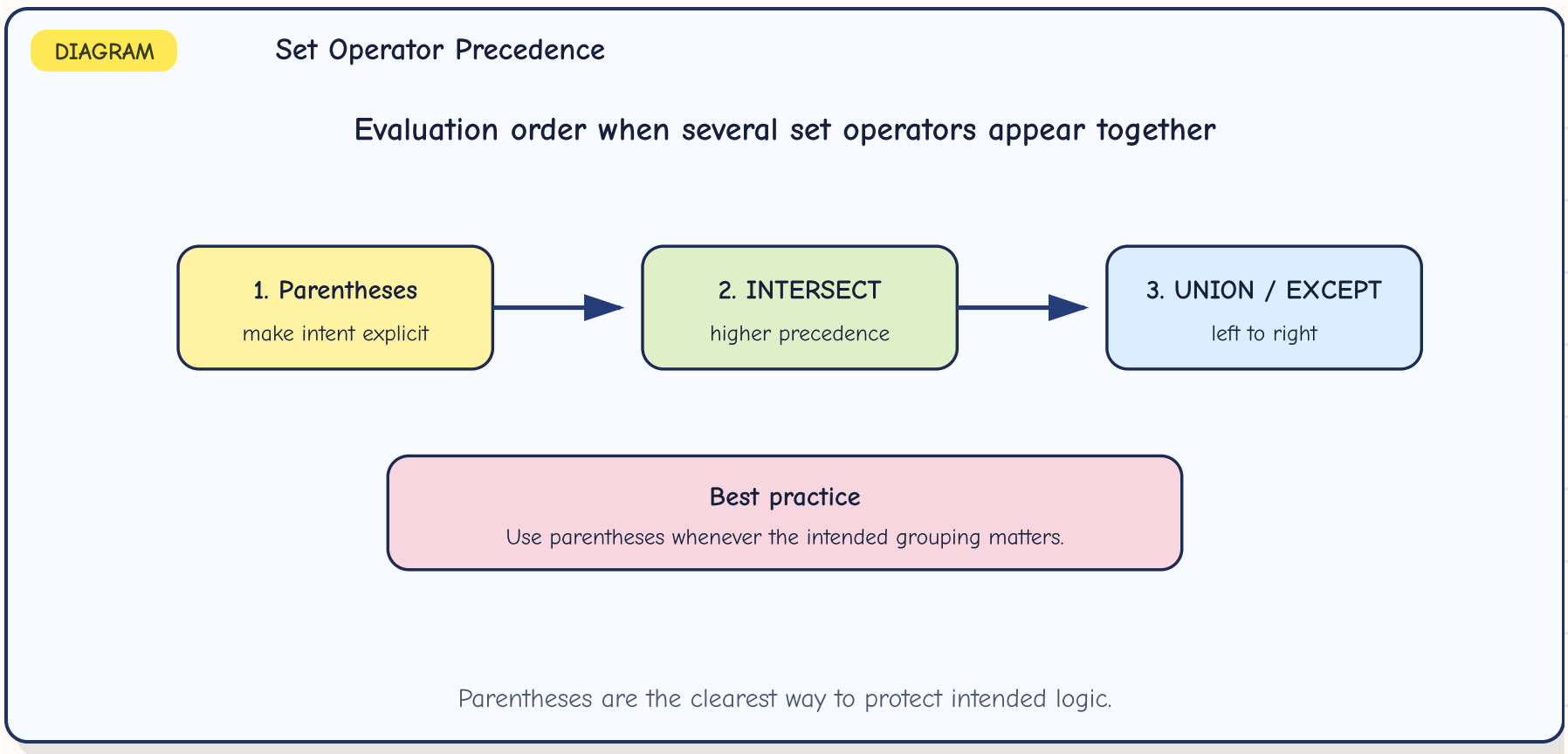
EXAMPLE / FLOW

```
Step 1
Combine Constituents with Gifts
or Interactions.

Step 2
Remove Constituents appearing in Task.
```

## Set Operator Precedence

DIAGRAM




SQL Server evaluates combined set operations in this order:

EXAMPLE / FLOW

```
1. Parentheses

2. INTERSECT

3. UNION and EXCEPT
   evaluated left to right
```

## Why Precedence Matters

Consider:

EXAMPLE / FLOW

```
SELECT ConstituentId
FROM fundraising.Gift

UNION

SELECT ConstituentId
FROM engagement.Interaction

INTERSECT

SELECT ConstituentId
FROM engagement.Task;
```

Because `INTERSECT` has higher precedence, SQL Server conceptually interprets it as:

EXAMPLE / FLOW

```
Gift

UNION

(
    Interaction
    INTERSECT
    Task
)
```

not:

EXAMPLE / FLOW

```
(
    Gift
    UNION
    Interaction
)

INTERSECT

Task
```

Those can produce very different results.

## Best Practice

If the logic matters:

> Use parentheses.

Do not force another developer to memorize precedence.

## Multiple UNION ALL Queries

You can combine more than two query results.

A good FundraisingDb example is a unified constituent activity stream.

EXAMPLE / FLOW

```
SELECT
    ConstituentId,

    CAST(GiftDate AS DATETIME2)
        AS ActivityDate,

    'Gift'
        AS ActivityType,

    CAST(TotalAmount AS VARCHAR(100))
        AS ActivityDetail

FROM fundraising.Gift

UNION ALL

SELECT
    ConstituentId,

    CAST(InteractionDate AS DATETIME2),

    'Interaction',

    Subject
```

EXAMPLE / FLOW (CONT. 2)

```
FROM engagement.Interaction

UNION ALL

SELECT
    ConstituentId,

    CAST(DueDate AS DATETIME2),

    'Task',

    Title

FROM engagement.Task

ORDER BY
    ActivityDate DESC;
```

## Why UNION ALL Is Correct for an Activity Feed

A Constituent can have:

EXAMPLE / FLOW

```
4 Gifts

10 Interactions

7 Tasks
```

Each one is a real record.

We want:

EXAMPLE / FLOW

```
21 activity rows
```

not a distinct set that might accidentally remove similar events.

Therefore:

EXAMPLE / FLOW

```
UNION ALL
```

is appropriate.

## Adding a Source Column

When combining different types of records, add a constant to identify the source.

Example:

EXAMPLE / FLOW

```
SELECT
    ConstituentId,
    GiftDate AS EventDate,
    'Gift' AS EventType
FROM fundraising.Gift

UNION ALL

SELECT
    ConstituentId,
    CAST(InteractionDate AS DATE),
    'Interaction'
FROM engagement.Interaction;
```

The new column:

EXAMPLE / FLOW

```
EventType
```

tells us which table produced the row.

## Why Constants Are Useful

Without:

EXAMPLE / FLOW

```
Gift

Interaction

Task
```

the final combined rows may become difficult to interpret.

Adding a source type makes a generic result set much easier to consume in:

EXAMPLE / FLOW

```
reports

APIs

timelines

dashboards
```

## Set Operations with Aggregates

You can combine aggregated results too.

Suppose we want one list containing:

EXAMPLE / FLOW

```
Campaign totals

Fund allocation totals
```

with the same output shape.

EXAMPLE / FLOW

```
SELECT
    CampaignId AS BusinessId,
    SUM(TotalAmount) AS Amount,
    'Campaign' AS BusinessType
FROM fundraising.Gift
WHERE CampaignId IS NOT NULL
GROUP BY CampaignId

UNION ALL

SELECT
    FundId,
    SUM(AllocationAmount),
    'Fund'
FROM fundraising.GiftAllocation
GROUP BY FundId;
```

Result shape:

EXAMPLE / FLOW

```
BusinessId
Amount
BusinessType
```

## Important Meaning of This Result

A Campaign amount means:

EXAMPLE / FLOW

```
Gift.TotalAmount grouped by Campaign.
```

A Fund amount means:

EXAMPLE / FLOW

```
GiftAllocation.AllocationAmount grouped by Fund.
```

They are conceptually different summaries.

The constant:

EXAMPLE / FLOW

```
BusinessType
```

makes that distinction explicit.

## UNION vs DISTINCT

These concepts are related but not identical.

`DISTINCT` removes duplicates from:

EXAMPLE / FLOW

```
one query
```

Example:

EXAMPLE / FLOW

```
SELECT DISTINCT
    ConstituentId
FROM fundraising.Gift;
```

`UNION` combines:

EXAMPLE / FLOW

```
two or more query results
```

and then gives distinct rows.

## Bad Use of UNION Instead of DISTINCT

Do not write:

EXAMPLE / FLOW

```
SELECT ConstituentId
FROM fundraising.Gift

UNION

SELECT ConstituentId
FROM fundraising.Gift;
```

just to remove duplicates.

Use:

EXAMPLE / FLOW

```
SELECT DISTINCT
    ConstituentId
FROM fundraising.Gift;
```

That expresses the intent correctly.

## INTERSECT vs INNER JOIN

These can sometimes answer similar questions.

Suppose we want:



Constituents who have both Gifts and Interactions.

Using `INTERSECT`:

EXAMPLE / FLOW

```
SELECT ConstituentId
FROM fundraising.Gift

INTERSECT

SELECT ConstituentId
FROM engagement.Interaction;
```

This is very clear because the requirement is:

EXAMPLE / FLOW

```
common IDs from two sets.
```

## Same Requirement Using JOIN

EXAMPLE / FLOW

```
SELECT DISTINCT
    g.ConstituentId
FROM fundraising.Gift AS g

INNER JOIN engagement.Interaction AS i
    ON i.ConstituentId = g.ConstituentId;
```

This can also work.

But if one Constituent has:

EXAMPLE / FLOW

```
5 Gifts
×
10 Interactions
```

the raw join may produce:

EXAMPLE / FLOW

```
50 rows
```

before distinct processing.

The set-oriented expression may be easier to understand.

## When JOIN Is Better

If you need columns from both sides:

EXAMPLE / FLOW

```
Gift information

Interaction information
```

use a JOIN.

`INTERSECT` is most natural when comparing equivalent result shapes.

## INTERSECT vs EXISTS

Another way to ask:

> Which Constituents have Gifts and Interactions?

EXAMPLE / FLOW

```
SELECT
    c.ConstituentId
FROM constituent.Constituent AS c

WHERE EXISTS
(
    SELECT 1
    FROM fundraising.Gift AS g
    WHERE g.ConstituentId = c.ConstituentId
)

AND EXISTS
(
    SELECT 1
    FROM engagement.Interaction AS i
    WHERE i.ConstituentId = c.ConstituentId
);
```

This is often natural when the main entity is already:

EXAMPLE / FLOW

```
Constituent
```

and you are checking related-row existence.

`INTERSECT` is elegant when directly comparing two equivalent sets.

## EXCEPT vs NOT EXISTS

Find Constituents with no Gifts.

Using `EXCEPT`:

EXAMPLE / FLOW

```
SELECT
    ConstituentId
FROM constituent.Constituent

EXCEPT

SELECT
    ConstituentId
FROM fundraising.Gift;
```

Using `NOT EXISTS`:

EXAMPLE / FLOW

```
SELECT
    c.ConstituentId
FROM constituent.Constituent AS c

WHERE NOT EXISTS
(
    SELECT 1
    FROM fundraising.Gift AS g
    WHERE g.ConstituentId = c.ConstituentId
);
```

Both can answer the requirement.

## When EXCEPT Is Excellent

Use **EXCEPT** when the problem sounds like:

EXAMPLE / FLOW

```
Give me set A
minus set B.
```

Examples:

EXAMPLE / FLOW

```
All Constituents
minus Gift Constituents

All Funds
minus allocated Funds

All Campaigns
minus Campaigns with Gifts
```

## When NOT EXISTS Is Often Better

Use `NOT EXISTS` when:

EXAMPLE / FLOW

```
the comparison is correlated,

you need additional relationship conditions,

or the business condition is naturally
phrased as "no related row exists."
```

## Set Operators with Different Schemas

Your FundraisingDb intentionally uses multiple schemas:

EXAMPLE / FLOW

```
core

constituent

fundraising

engagement
```

That does not prevent set operations.

Example:

EXAMPLE / FLOW

```
SELECT
    Email
FROM core.AppUser
WHERE Email IS NOT NULL

UNION

SELECT
    PrimaryEmail
FROM constituent.Constituent
WHERE PrimaryEmail IS NOT NULL;
```

This creates a unique email list across:

EXAMPLE / FLOW

```
application users

and

constituents.
```

## UNION ALL Email Example

If instead you need every record and want to know the source:

EXAMPLE / FLOW

```
SELECT
    Email AS EmailAddress,
    'AppUser' AS ContactType
FROM core.AppUser
WHERE Email IS NOT NULL

UNION ALL

SELECT
    PrimaryEmail,
    'Constituent'
FROM constituent.Constituent
WHERE PrimaryEmail IS NOT NULL;
```

Now the same email may appear multiple times if it belongs to multiple records.

That may be correct.

## Data Type Precedence and Explicit CAST

Suppose you combine:

EXAMPLE / FLOW

```
GiftId

InteractionId

TaskId
```

into a common:

EXAMPLE / FLOW

```
ActivityId
```

If all IDs are the same integer type, no problem.

If types differ:

EXAMPLE / FLOW

```
BIGINT

INT
```

SQL Server determines the compatible result type according to data-type precedence.

For clarity, you can write:

EXAMPLE / FLOW

```
SELECT
    CAST(GiftId AS BIGINT) AS ActivityId
FROM fundraising.Gift

UNION ALL

SELECT
    CAST(InteractionId AS BIGINT)
FROM engagement.Interaction;
```

## String Length and Precision

Corresponding columns may differ in:

EXAMPLE / FLOW

```
VARCHAR length

DECIMAL precision

DECIMAL scale
```

SQL Server determines a resulting type according to its normal type-combination rules.

Do not assume the second query determines the final data type.

## Collation Considerations

Character data from different databases or differently collated columns may have incompatible collations.

In such cases SQL Server may require an explicit:

EXAMPLE / FLOW

```
COLLATE
```

expression.

Within one well-designed FundraisingDb, this is less likely if related string columns use consistent collations.

## INTERSECT and EXCEPT Data-Type Restrictions

`INTERSECT` and `EXCEPT` require comparable data types.

SQL Server does not directly support comparing some types through these operators, including:

EXAMPLE / FLOW

```
xml

text

ntext

image

nonbinary CLR user-defined types
```

for direct result comparison.

Legacy types such as:

EXAMPLE / FLOW

```
text
ntext
image
```

should generally not be used in new SQL Server development anyway.

## Practical Pattern 1 - All Engaged Constituents

Requirement:

> Return unique Constituents who either made a Gift, had an Interaction, or have a Task.

EXAMPLE / FLOW

```
SELECT
    ConstituentId
FROM fundraising.Gift

UNION

SELECT
    ConstituentId
FROM engagement.Interaction

UNION

SELECT
    ConstituentId
FROM engagement.Task;
```

Because `UNION` is used:

EXAMPLE / FLOW

```
one Constituent appears once
```

regardless of how many business activities exist.

## Practical Pattern 2 - Complete Activity Timeline

Requirement:

> Return every Gift, Interaction, and Task for a Constituent timeline.

Use:

EXAMPLE / FLOW

```
UNION ALL
```

because every activity is important.

EXAMPLE / FLOW

```
SELECT
    ConstituentId,
    CAST(GiftDate AS DATETIME2) AS ActivityDate,
    'Gift' AS ActivityType,
    CAST(TotalAmount AS VARCHAR(200)) AS Detail
FROM fundraising.Gift

UNION ALL

SELECT
    ConstituentId,
    CAST(InteractionDate AS DATETIME2),
    'Interaction',
    Subject
FROM engagement.Interaction

UNION ALL

SELECT
    ConstituentId,
    CAST(DueDate AS DATETIME2),
    'Task',
    Title
FROM engagement.Task

ORDER BY ActivityDate DESC;
```

## Practical Pattern 3 - Constituents Who Both Donated and Interacted

EXAMPLE / FLOW

```
SELECT
    ConstituentId
FROM fundraising.Gift

INTERSECT

SELECT
    ConstituentId
FROM engagement.Interaction;
```

Meaning:

EXAMPLE / FLOW

```
Gift set
AND
Interaction set
```

## Practical Pattern 4 - Prospects Who Have Never Donated

EXAMPLE / FLOW

```
SELECT
    ConstituentId
FROM constituent.Constituent

EXCEPT

SELECT
    ConstituentId
FROM fundraising.Gift;
```

Meaning:

EXAMPLE / FLOW

```
All Constituents
minus
donors.
```

## Practical Pattern 5 - Funds Never Used

EXAMPLE / FLOW

```
SELECT
    FundId
FROM fundraising.Fund

EXCEPT

SELECT
    FundId
FROM fundraising.GiftAllocation;
```

Meaning:

EXAMPLE / FLOW

```
Funds existing in the system
that have never received an allocation.
```

## Practical Pattern 6 - Campaigns with Donors Who Also Have Tasks

First set:

EXAMPLE / FLOW

```
Constituents who donated to Campaign 200
```

Second set:

EXAMPLE / FLOW

```
Constituents who have Tasks
```

EXAMPLE / FLOW

```
SELECT
    ConstituentId
FROM fundraising.Gift
WHERE CampaignId = 200

INTERSECT

SELECT
    ConstituentId
FROM engagement.Task;
```

## Practical Pattern 7 - All Staff Participation IDs

Suppose we want AppUsers involved in:

EXAMPLE / FLOW

```
Interactions

Task creation

Task assignment
```

EXAMPLE / FLOW

```
SELECT
    FundraiserUserId AS AppUserId
FROM engagement.Interaction

UNION

SELECT
    CreatedByUserId
FROM engagement.Task

UNION

SELECT
    AssignedToUserId
FROM engagement.Task;
```

This returns unique AppUser IDs involved in those activities.

## Practical Pattern 8 - Preserve Every Staff Activity

If we want the number of staff activities instead:

EXAMPLE / FLOW

```
SELECT
    FundraiserUserId AS AppUserId,
    'Interaction' AS ActivityType
FROM engagement.Interaction

UNION ALL

SELECT
    CreatedByUserId,
    'TaskCreated'
FROM engagement.Task

UNION ALL

SELECT
    AssignedToUserId,
    'TaskAssigned'
FROM engagement.Task;
```

Then aggregate:

EXAMPLE / FLOW

```
WITH StaffActivity AS
(
    SELECT
        FundraiserUserId AS AppUserId,
        'Interaction' AS ActivityType
    FROM engagement.Interaction

    UNION ALL

    SELECT
        CreatedByUserId,
        'TaskCreated'
    FROM engagement.Task

    UNION ALL

    SELECT
        AssignedToUserId,
        'TaskAssigned'
    FROM engagement.Task
)
```

EXAMPLE / FLOW (CONT. 2)

```
SELECT
    AppUserId,
    COUNT(*) AS TotalActivityCount
FROM StaffActivity
GROUP BY AppUserId;
```

## Common Set Operator Mistakes

### Mistake 1 - Different Column Counts

Wrong:

EXAMPLE / FLOW

```
SELECT
    ConstituentId,
    DisplayName
FROM constituent.Constituent

UNION

SELECT
    ConstituentId
FROM fundraising.Gift;
```

The branches do not return the same number of columns.

## Mistake 2 - Wrong Column Order

Wrong:

EXAMPLE / FLOW

```
SELECT
    AppUserId,
    DisplayName
FROM core.AppUser

UNION ALL

SELECT
    DisplayName,
    ConstituentId
FROM constituent.Constituent;
```

Set operations use positional matching.

## Mistake 3 - Incompatible Types

Do not combine:

EXAMPLE / FLOW

```
BIGINT ID
```

with:

EXAMPLE / FLOW

```
arbitrary text name
```

in the corresponding position.

Use logically compatible columns.

## Mistake 4 - Using UNION Automatically

Many developers write:

EXAMPLE / FLOW

```
UNION
```

every time.

But duplicate removal costs work.

If every row should be preserved, use:

EXAMPLE / FLOW

```
UNION ALL.
```

## Mistake 5 - Using UNION ALL When Duplicates Are Wrong

Suppose the question is:

> Which Constituents have any engagement?

You probably want each Constituent once.

Using `UNION ALL` could produce:

EXAMPLE / FLOW

```
1001
1001
1001
1001
...
```

Use `UNION` when uniqueness is part of the requirement.

## Mistake 6 - Forgetting EXCEPT Direction

Remember:

EXAMPLE / FLOW

```
A EXCEPT B
```

means:

EXAMPLE / FLOW

```
in A
but not B.
```

The order matters.

## Mistake 7 - Expecting INTERSECT to Preserve Duplicates

It does not.

`INTERSECT` returns distinct common rows.

## Mistake 8 - Forgetting Whole-Row Comparison

This:

EXAMPLE / FLOW

```
SELECT
    ConstituentId,
    CampaignId
FROM fundraising.Gift
```

compares both columns.

Rows:

EXAMPLE / FLOW

```
1001 | 200

1001 | 201
```

are different.

## Mistake 9 - Putting ORDER BY in Each Top-Level Branch

Use one final:

EXAMPLE / FLOW

```
ORDER BY
```

for the combined result.

## Mistake 10 - Using Set Operators When a JOIN Is Needed

Question:

Show every Gift and its Constituent name.

Use:

EXAMPLE / FLOW

```
JOIN
```

not:

EXAMPLE / FLOW

```
UNION.
```

Question:

Combine AppUser email addresses and Constituent email addresses.

Use:

EXAMPLE / FLOW

```
UNION
or
UNION ALL.
```

## Set Operator Performance Basics

Performance depends on:

EXAMPLE / FLOW

```
number of rows

row width

indexes

filtering

data types

sorting

hashing

memory grants

statistics

execution plan
```

The operator itself is only part of the story.

## UNION ALL Performance

`UNION ALL` usually has the least duplicate-processing overhead because SQL Server can concatenate input rows without eliminating duplicates.

## UNION Performance

`UNION` requires distinct output.

That may result in:

EXAMPLE / FLOW

```
Sort

Hash Match

other distinct processing
```

depending on the execution plan.

## INTERSECT Performance

SQL Server must determine:

EXAMPLE / FLOW

```
which complete rows exist in both sets
```

and return distinct rows.

Good indexing and smaller filtered inputs can help.

## EXCEPT Performance

SQL Server must determine:

EXAMPLE / FLOW

```
which distinct left-side rows
do not exist on the right side.
```

Again, performance depends on the actual query and plan.

## Filter Before the Set Operation When Appropriate

Suppose we only need 2026 engagement activity.

Good clear query:

EXAMPLE / FLOW

```
SELECT
    ConstituentId,
    GiftDate AS ActivityDate
FROM fundraising.Gift
WHERE GiftDate >= '2026-01-01'
  AND GiftDate <  '2027-01-01'

UNION ALL

SELECT
    ConstituentId,
    CAST(InteractionDate AS DATE)
FROM engagement.Interaction
WHERE InteractionDate >= '2026-01-01'
  AND InteractionDate <  '2027-01-01';
```

Each branch clearly restricts itself to the required period.

## Avoid Unnecessary Wide Rows

If you only need:

EXAMPLE / FLOW

```
ConstituentId
```

do not return:

EXAMPLE / FLOW

```
every Constituent column

every Gift column

every Interaction column
```

Set operations become more expensive as the rows being compared become wider.

## Choose UNION ALL When Business Logic Guarantees Separation

Suppose you combine:

EXAMPLE / FLOW

```
Gift events

Interaction events

Task events
```

with a source-type column.

These represent different entity types.

Duplicate elimination normally provides no business value.

Therefore:

EXAMPLE / FLOW

```
UNION ALL
```

is generally appropriate for the timeline.

## Set Operators Inside a CTE

Set operators and CTEs work very well together.

Example:

EXAMPLE / FLOW

```
WITH ConstituentActivity AS
(
    SELECT
        ConstituentId,
        CAST(GiftDate AS DATETIME2) AS ActivityDate,
        'Gift' AS ActivityType
    FROM fundraising.Gift

    UNION ALL

    SELECT
        ConstituentId,
        CAST(InteractionDate AS DATETIME2),
        'Interaction'
    FROM engagement.Interaction

    UNION ALL

    SELECT
        ConstituentId,
        CAST(DueDate AS DATETIME2),
        'Task'
    FROM engagement.Task
)
```

EXAMPLE / FLOW (CONT. 2)

```
SELECT
    ConstituentId,
    COUNT(*) AS ActivityCount
FROM ConstituentActivity
GROUP BY ConstituentId;
```

This is a realistic combination of topics you have already studied:

EXAMPLE / FLOW

```
CTE

UNION ALL

GROUP BY

COUNT
```

## Decision Guide

Ask:

> Do I need to combine rows from several equivalent result sets?

If yes:

EXAMPLE / FLOW

```
Should duplicate rows remain?

YES
→ UNION ALL

NO
→ UNION
```

Ask:

> Do I need only rows that appear in both result sets?

Use:

EXAMPLE / FLOW

```
INTERSECT
```

Ask:

> Do I need rows from the first result that do not appear in the second?

Use:

EXAMPLE / FLOW

```
EXCEPT
```

Ask:

Do I need columns from related rows side by side?

Use:

EXAMPLE / FLOW

`JOIN`

## Master Set Operator Table

| Requirement | Operator |
| --- | --- |
| Combine unique rows | `UNION` |
| Combine every row | `UNION ALL` |
| Find common rows | `INTERSECT` |
| Find left-side differences | `EXCEPT` |
| Combine related columns | `JOIN`, not a set operator |

## Master Rules

EXAMPLE / FLOW

Same number of columns

EXAMPLE / FLOW

Corresponding columns in same order

EXAMPLE / FLOW

Compatible data types

EXAMPLE / FLOW

Columns matched by position

EXAMPLE / FLOW

Final names come from first query

EXAMPLE / FLOW

UNION removes duplicates

EXAMPLE / FLOW

UNION ALL preserves duplicates

EXAMPLE / FLOW

INTERSECT returns distinct common rows

EXAMPLE / FLOW

EXCEPT returns distinct left-only rows

EXAMPLE / FLOW

Final ORDER BY belongs at the end

EXAMPLE / FLOW

```
INTERSECT has higher precedence
than UNION and EXCEPT
```

EXAMPLE / FLOW

```
Use parentheses when combining operators
to make intended logic obvious
```

## 20 Interview Questions and Query Problems

## Interview Question 1

### What are set operators in SQL Server?

Set operators combine or compare complete query results.

The main operators are:

EXAMPLE / FLOW

```
UNION
UNION ALL
INTERSECT
EXCEPT
```

They are useful when multiple SELECT queries return compatible result shapes.

## Interview Question 2

### What requirements must two queries satisfy before using a set operator?

They must return:

EXAMPLE / FLOW

```
the same number of columns,

columns in corresponding positions,

and compatible data types.
```

Column names do not need to be identical.

SQL Server combines them according to position rather than matching names.

## Interview Question 3

### What is the difference between UNION and UNION ALL?

UNION:

EXAMPLE / FLOW

```
combines rows
and removes duplicate complete rows.
```

UNION ALL:

EXAMPLE / FLOW

```
combines rows
and preserves every row.
```

If duplicate removal is unnecessary, UNION ALL usually requires less processing.

## Interview Question 4 - Query Problem

**Return all unique Constituents who either donated or had an Interaction.**

EXAMPLE / FLOW

```
SELECT
    ConstituentId
FROM fundraising.Gift

UNION

SELECT
    ConstituentId
FROM engagement.Interaction;
```

### Explanation

Gift contributes:

EXAMPLE / FLOW

```
donors
```

Interaction contributes:

EXAMPLE / FLOW

```
engaged Constituents
```

`UNION` removes repeated Constituent IDs.

Therefore each Constituent appears once.

## Interview Question 5 - Query Problem

**Build a combined Gift and Interaction activity stream without losing duplicate events.**

EXAMPLE / FLOW

```
SELECT
    ConstituentId,
    CAST(GiftDate AS DATETIME2) AS ActivityDate,
    'Gift' AS ActivityType
FROM fundraising.Gift

UNION ALL

SELECT
    ConstituentId,
    CAST(InteractionDate AS DATETIME2),
    'Interaction'
FROM engagement.Interaction

ORDER BY ActivityDate DESC;
```

### Why UNION ALL?

Every Gift and every Interaction is an independent business record.

Removing duplicates could incorrectly remove legitimate activity.

## Interview Question 6

### How does UNION decide whether two rows are duplicates?

It compares the complete selected row.

Example:

EXAMPLE / FLOW

```
1001 | Gift
```

and:

EXAMPLE / FLOW

```
1001 | Interaction
```

are not duplicates because the second column differs.

## Interview Question 7

### What is INTERSECT?

`INTERSECT` returns distinct rows appearing in both query results.

It represents:

EXAMPLE / FLOW

```
A AND B
```

or:

EXAMPLE / FLOW

```
A ∩ B.
```

## Interview Question 8 - Query Problem

### Find Constituents who have both Gifts and Interactions.

EXAMPLE / FLOW

```
SELECT
    ConstituentId
FROM fundraising.Gift

INTERSECT

SELECT
    ConstituentId
FROM engagement.Interaction;
```

### Explanation

Only IDs present in both result sets survive.

A Constituent with many Gifts and many Interactions still appears once.

## Interview Question 9

### What is EXCEPT?

EXCEPT returns distinct rows from the first query that are absent from the second query.

Think:

EXAMPLE / FLOW

```
A - B
```

The order of the queries matters.

## Interview Question 10 - Query Problem

Find Constituents who have never made a Gift.

EXAMPLE / FLOW

```
SELECT
    ConstituentId
FROM constituent.Constituent

EXCEPT

SELECT
    ConstituentId
FROM fundraising.Gift;
```

### Explanation

The first set is:

EXAMPLE / FLOW

```
all Constituents.
```

The second set is:

EXAMPLE / FLOW

```
Constituents who donated.
```

Removing the second from the first leaves:

EXAMPLE / FLOW

```
Constituents with no Gifts.
```

## Interview Question 11 - Query Problem

Find Funds that have never received an allocation.

EXAMPLE / FLOW

```
SELECT
    FundId
FROM fundraising.Fund

EXCEPT

SELECT
    FundId
FROM fundraising.GiftAllocation;
```

### Explanation

The first query returns all Funds.

The second returns Funds used by GiftAllocation.

EXCEPT returns unused Funds.

## Interview Question 12

### Why is A EXCEPT B different from B EXCEPT A?

Because EXCEPT is directional.

EXAMPLE / FLOW

```
A EXCEPT B
=
rows only in A.
```

EXAMPLE / FLOW

```
B EXCEPT A
=
rows only in B.
```

They answer different questions.

## Interview Question 13

### How are NULL values handled by INTERSECT and EXCEPT?

When SQL Server determines distinct rows for these set operators, two NULL values are considered equal.

This is different from ordinary comparison logic such as:

EXAMPLE / FLOW

```
NULL = NULL
```

which does not evaluate to true.

## Interview Question 14

### Where should ORDER BY be placed when using UNION, INTERSECT, or EXCEPT?

Normally at the end of the complete top-level set expression.

Example:

EXAMPLE / FLOW

```
SELECT
    ConstituentId,
    GiftDate AS ActivityDate
FROM fundraising.Gift

UNION ALL

SELECT
    ConstituentId,
    CAST(InteractionDate AS DATE)
FROM engagement.Interaction

ORDER BY ActivityDate DESC;
```

The final ORDER BY sorts the complete combined result.

## Interview Question 15

What is the precedence of SQL Server set operators?

SQL Server uses:

EXAMPLE / FLOW

```
1. Parentheses

2. INTERSECT

3. UNION and EXCEPT,
   evaluated left to right
```

Use parentheses whenever the intended logic is important.

## Interview Question 16 - Query Problem

Find unique AppUsers who participated in Interactions or Tasks.

EXAMPLE / FLOW

```
SELECT
    FundraiserUserId AS AppUserId
FROM engagement.Interaction

UNION

SELECT
    AssignedToUserId
FROM engagement.Task

UNION

SELECT
    CreatedByUserId
FROM engagement.Task;
```

### Explanation

The three sets represent:

EXAMPLE / FLOW

```
fundraisers on Interactions,

assigned Task users,

Task creators.
```

**UNION** produces one unique list of AppUsers involved in those activities.

## Interview Question 17 - Query Problem

Create one activity stream containing Gifts, Interactions, and Tasks.

EXAMPLE / FLOW

```
SELECT
    ConstituentId,
    CAST(GiftDate AS DATETIME2) AS ActivityDate,
    'Gift' AS ActivityType,
    CAST(TotalAmount AS VARCHAR(200)) AS Description
FROM fundraising.Gift

UNION ALL

SELECT
    ConstituentId,
    CAST(InteractionDate AS DATETIME2),
    'Interaction',
    Subject
FROM engagement.Interaction

UNION ALL

SELECT
    ConstituentId,
    CAST(DueDate AS DATETIME2),
    'Task',
    Title
FROM engagement.Task

ORDER BY ActivityDate DESC;
```

## Explanation

All three queries have the same shape:

EXAMPLE / FLOW

```
ConstituentId

ActivityDate

ActivityType

Description
```

`UNION ALL` preserves every real event.

## Interview Question 18 — What is the difference between INTERSECT and INNER JOIN?

`INTERSECT` compares equivalent result sets and returns distinct rows common to both.

`INNER JOIN` matches related rows using a join predicate and can return columns from both tables.

Use `INTERSECT` when the requirement is naturally:

EXAMPLE / FLOW

```
Which values are common to both sets?
```

Use `JOIN` when you need relationship details from both sides.

## Interview Question 19 — What is the difference between EXCEPT and NOT EXISTS?

Both can express anti-match logic.

`EXCEPT` is set-oriented:

EXAMPLE / FLOW

```
A minus B.
```

`NOT EXISTS` is relationship-oriented:

EXAMPLE / FLOW

```
Return this row when no matching
related row exists.
```

Example using `NOT EXISTS`:

EXAMPLE / FLOW

```
SELECT
    c.ConstituentId,
    c.DisplayName
FROM constituent.Constituent AS c

WHERE NOT EXISTS
(
    SELECT 1
    FROM fundraising.Gift AS g
    WHERE g.ConstituentId = c.ConstituentId
);
```

`NOT EXISTS` is often easier when extra correlated conditions are required.

## Interview Question 20 - Complete Problem

Requirement

Create a report containing Constituents who:

EXAMPLE / FLOW

```
made a Gift

OR

had an Interaction
```

but exclude Constituents who currently have a Task.

Return the Constituent name.

### Step 1 - Gift Constituents

EXAMPLE / FLOW

```
SELECT
    ConstituentId
FROM fundraising.Gift
```

### Step 2 - Add Interaction Constituents

EXAMPLE / FLOW

```
SELECT
    ConstituentId
FROM fundraising.Gift

UNION

SELECT
    ConstituentId
FROM engagement.Interaction
```

This produces:

EXAMPLE / FLOW

```
Constituents with Gift OR Interaction.
```

### Step 3 - Remove Constituents with Tasks

EXAMPLE / FLOW

```
(
    SELECT
        ConstituentId
    FROM fundraising.Gift

    UNION

    SELECT
        ConstituentId
    FROM engagement.Interaction
)

EXCEPT

SELECT
    ConstituentId
FROM engagement.Task;
```

### Step 4 - Add the Constituent Name

Use the set result inside a CTE:

EXAMPLE / FLOW

```
WITH TargetConstituents AS
(
    (
        SELECT
            ConstituentId
        FROM fundraising.Gift

        UNION

        SELECT
            ConstituentId
        FROM engagement.Interaction
    )

    EXCEPT

    SELECT
        ConstituentId
    FROM engagement.Task
)

SELECT
    c.ConstituentId,
    c.DisplayName
```

EXAMPLE / FLOW (CONT. 2)

```
FROM TargetConstituents AS t

INNER JOIN constituent.Constituent AS c
    ON c.ConstituentId = t.ConstituentId

ORDER BY
    c.DisplayName;
```

## Understand the Complete Logic

EXAMPLE / FLOW

```
Gift Constituents
        \
         UNION
        /
Interaction Constituents
        |
        v
Unique engaged Constituents
        |
      EXCEPT
        |
Task Constituents
        |
        v
Engaged Constituents
without Tasks
        |
        v
JOIN Constituent
        |
        v
Display names
```

This example combines topics you have already learned:

EXAMPLE / FLOW

```
CTE

UNION

EXCEPT

JOIN

ORDER BY
```

## Final Revision Map

EXAMPLE / FLOW

```
SET OPERATORS
        |
        +-------------------------+
        |                         |
      COMBINE                   COMPARE
        |                         |
  UNION / UNION ALL       INTERSECT / EXCEPT
```

EXAMPLE / FLOW

```
UNION

Gift Constituents
+
Interaction Constituents

→ Unique combined IDs
```

EXAMPLE / FLOW

```
UNION ALL

Gift events
+
Interaction events
+
Task events

→ Keep every activity
```

EXAMPLE / FLOW

```
INTERSECT

Gift Constituents
AND
Interaction Constituents

→ Constituents in both
```

EXAMPLE / FLOW

```
EXCEPT

All Constituents
MINUS
Gift Constituents

→ Constituents with no Gifts
```

## Final Easy Memory Rule

EXAMPLE / FLOW

```
UNION
→ A OR B, duplicates removed

UNION ALL
→ A + B, everything preserved

INTERSECT
→ A AND B

EXCEPT
→ A BUT NOT B
```

And always remember the structural rule:

EXAMPLE / FLOW

```
Same number of columns
+
Correct column order
+
Compatible data types
```

The most important business rule is:

Choose the set operator based on what the rows mean. Use **UNION** when you need one unique combined set, **UNION ALL** when every business record must survive, **INTERSECT** when you want records common to both sets, and **EXCEPT** when you want the first set minus the second.